

EEC 277
Digital Signal Processing

Final Project Report

AUTHORS

Andrew Sameh Adel Mikhail (G3 S1) – ID: 9489

Mazen Elkaslan (G2 S1) – ID: 9445

Galal Mohamed Taman (G3 S1) – ID: 9453

Throughout this report, the two recommended samples have been used. This includes **Record 100** and **Record 106** from PhysioNet database.

Part 1:

[1] Problem Definitions and Specifications

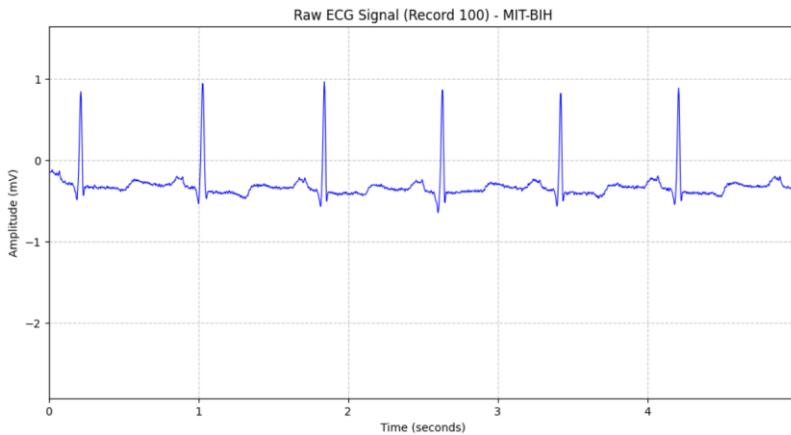


Fig 1.1: Record 100 ECG Signal with normal sinus rhythm

The flat segments in between each spike are misaligned from the 0mV axis, this is called the **Baseline Wander**.

It is a relatively clean signal, but one that has slight noise visible in the trace signal.

This signal shows a similar rhythm, but it also features **PVCs (Premature Ventricular Contractions)**. These are the wider taller peaks after the larger spikes in each period.

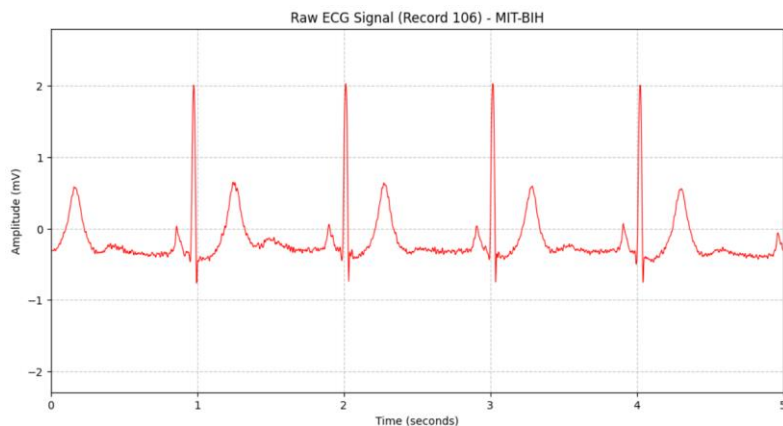


Fig 1.2: Record 106 ECG Signal with abnormal ventricular ectopic beats

If looked closely, there is more visible thickness within the flat segments that suggests a higher level of **power-line interference**. That is indicated by the subtle oscillations as you zoom in in the flat segments.

Considering these characteristics, the filtering objectives that need to be completed are (1) adjusting the baseline wander back to the 0mV axis, (2) removing the power-line interference within the abnormal signal and (3) filtering any noise within T- and P- waves and the flat segments.

This will be achieved using 3 filters cascaded in series after one another:

1. **High-Pass Filter:** To remove baseline wander. It allows everything above 0.5Hz to pass.
2. **Notch Filter:** Tuned at 50Hz to remove the power-line interference while leaving surrounding frequencies untouched.
3. **Low-Pass Filter:** To remove the **muscle** noise and smoothen the signal.

Filter Specifications

When implementing filters, one must consider its properties to attune the filter's effect to get the desired results. Here are 4 properties for the filters used set in the recommended constraints along with a reason why it was set as such:

	Condition	Justification
Passband Ripple	Very low ripple	If passband ripple is high, it can distort height of the R-peaks or the depth of the S-waves. Can lead to misdiagnosis.
Stopband Attenuation	$40dB < A_s < 60dB$	Reduce 50Hz power-line noise to 1% of its original amplitude, to make flat segments sharper.
Transition Bandwidth	Narrow	Remove exactly the 50Hz without deleting the 48Hz or 52Hz components of QRS complex.
Cutoff Frequencies	HPF = 0.5Hz,	Remove respiration artifacts.
	LPF = 100Hz,	Preserve sharp peaks of QRS complex.
	Notch = 50Hz	Standard.

Definitions:

- **Passband Ripple (R_p):** The maximum allowable fluctuation in the frequencies we want to keep.
- **Stopband Attenuation (A_s):** How much the unwanted noise is weakened.
- **Transition Bandwidth (Δf):** The speed at which the filter switches from passing to stopping.

Realistic Implementation Constraints

There are a few constraints that need to be considered when these filters are implemented in realistic situations. Here are 4 constraints as suggested in the requirements:

Constraint	
Maximum Filter Order	More memory and processing power is needed, the higher the order of the filter is. We generally use lower orders.
Computational Complexity	Every multiplication in the filter consumes battery life.
Latency	The time it takes for a heartbeat to go through the filter and appear on a screen.
Phase Distortion	Most IIR filters shift different frequencies by different amounts of time, therefore we could use Linear Phase FIR filter to ensure the waves stay where they belong in time.

[2] Filter Design and Analysis

For the 3 filter types mentioned above, each filter will be assigned a specific digital filter to carry out its role. Details about each digital filter are listed below:

1. High-Pass Filter:

- Uses **IIR Butterworth**, because it has a maximally flat passband.

Sampling frequency (F_s)	360Hz
Cutoff frequency (F_c)	0.5Hz
Filter order	4

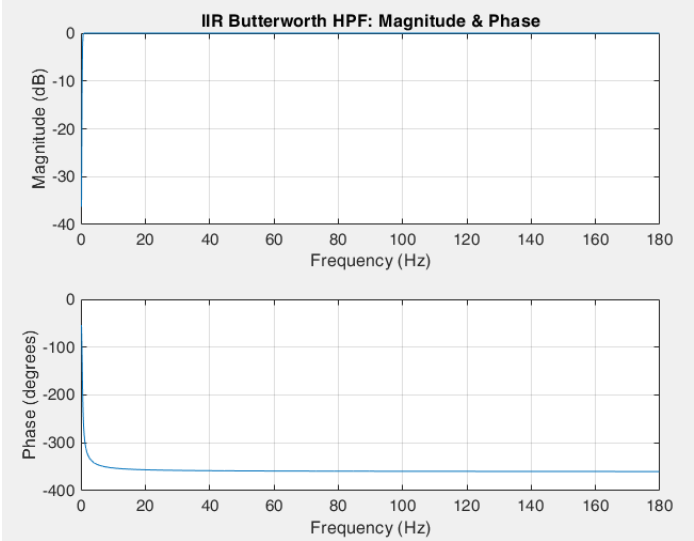


Fig 2.1: Magnitude and Phase Response for Butterworth

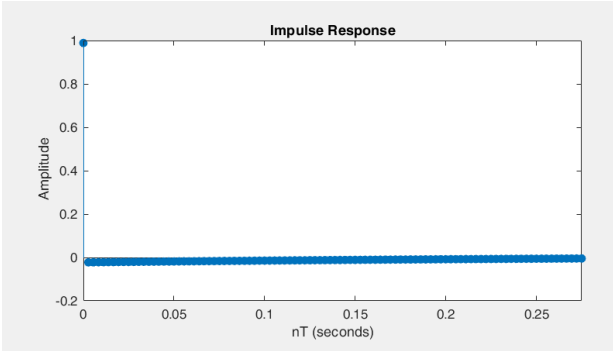


Fig 2.2: Impulse Response for Butterworth

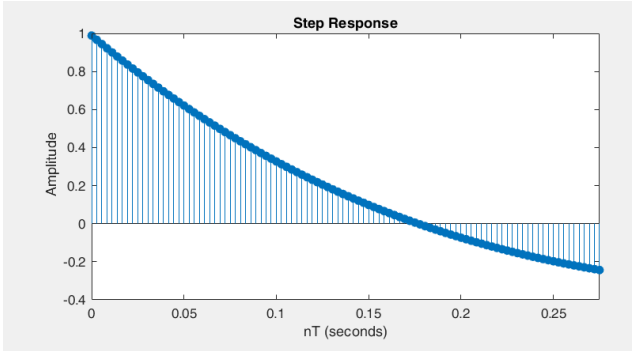


Fig 2.3: Step Response for Butterworth

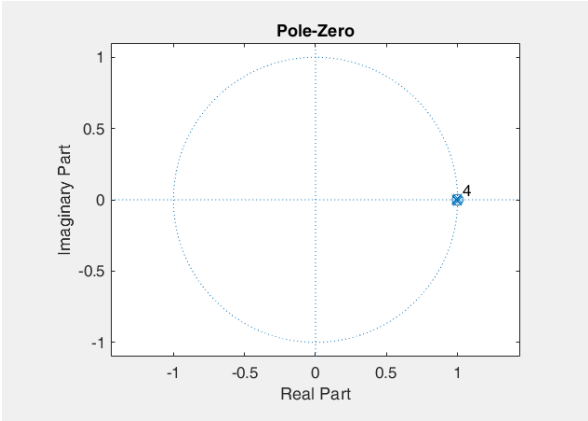


Fig 2.4: Pole-zero diagram for Butterworth

MATLAB Code:

```
fs = 360;           % Sampling frequency
fc = 0.5;           % Cutoff frequency
order = 4;          % Filter order

[b, a] = butter(order, fc/(fs/2), 'high');

figure;
freqz(b, a, 1024, fs);
title('IIR Butterworth HPF: Magnitude & Phase');

figure;
subplot(2,2,1);
zplane(b, a);
title('Pole-Zero');

subplot(2,2,2);
impz(b, a, 100, fs);
title('Impulse Response');

subplot(2,2,3);
stepz(b, a, 100, fs);
title('Step Response');
```

2. Notch Filter:

- Uses **Chebyshev Type II**, because it allows for a flat passband while placing the unneeded noise in the 50Hz band that is filtered out.

Sampling frequency (F_s)	360Hz
Notch frequency (F_n)	50Hz
Bandwidth	2

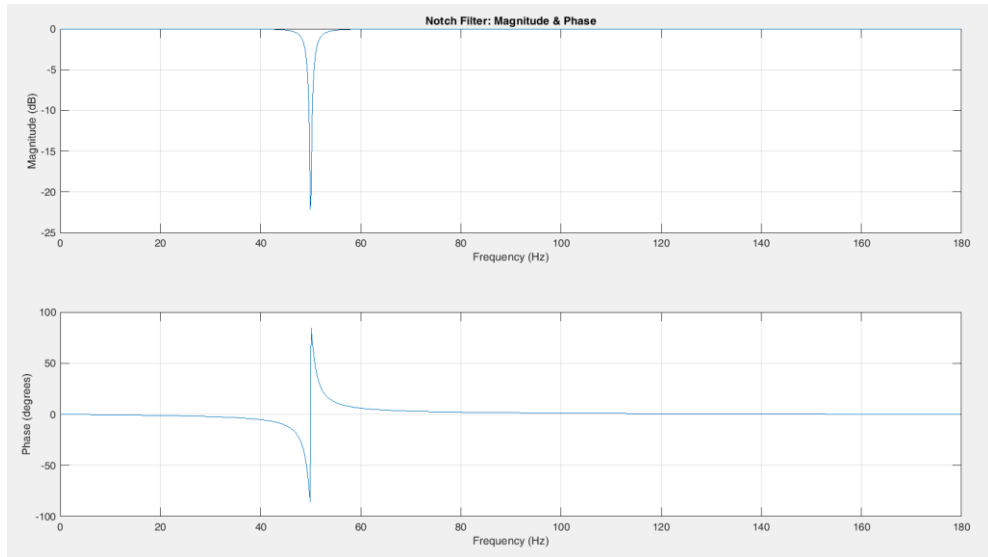


Fig 2.5: Magnitude and Phase Response for Chebyshev Type II

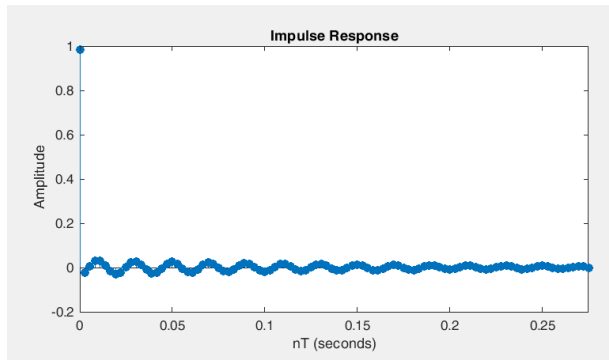


Fig 2.6: Impulse Response for Chebyshev Type II

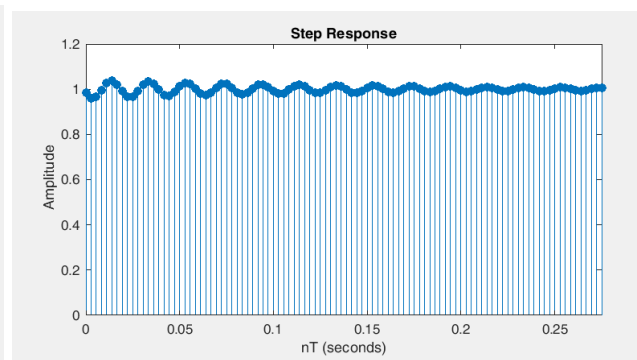


Fig 2.7: Step Response for Chebyshev Type II

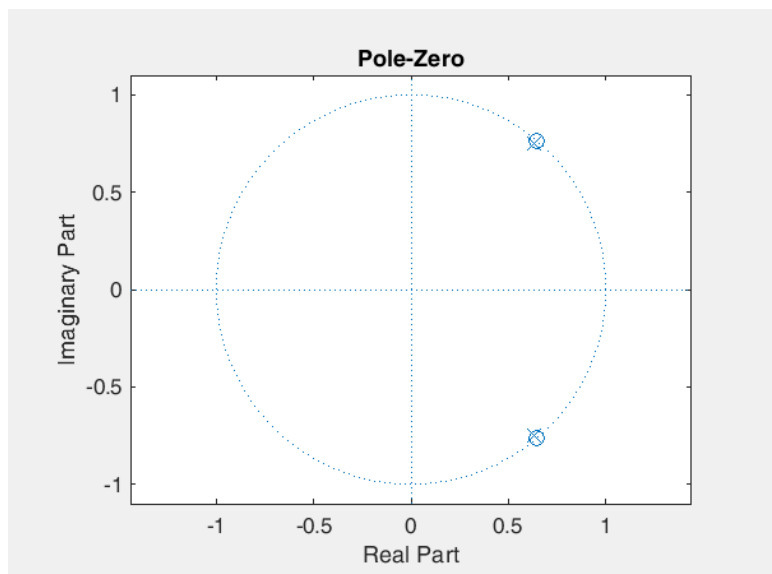


Fig 2.8: Pole-zero diagram for Chebyshev Type II

MATLAB Code:

```
fs = 360;           % Sampling frequency
f_notch = 50;       % Frequency of the notch
bw = 2;             % Bandwidth of the notch
[b, a] = iirnotch(f_notch/(fs/2), bw/(fs/2));

% Analysis
figure;
freqz(b, a, 1024, fs);
title('Notch Filter: Magnitude & Phase');

figure;
subplot(2,2,1);
zplane(b, a);
title('Pole-Zero');

subplot(2,2,2);
impz(b, a, 100, fs);
title('Impulse Response');

subplot(2,2,3);
stepz(b, a, 100, fs);
title('Step Response');
```

3. Low Pass Filter:

- Uses **FIR Window-based filter**, because it provides linear phase which ensures that all waves are delayed equally to prevent smearing the QRS complex.

Sampling frequency (F_s)	360Hz
Cutoff frequency (F_c)	100Hz
Stopband Attenuation	50dB

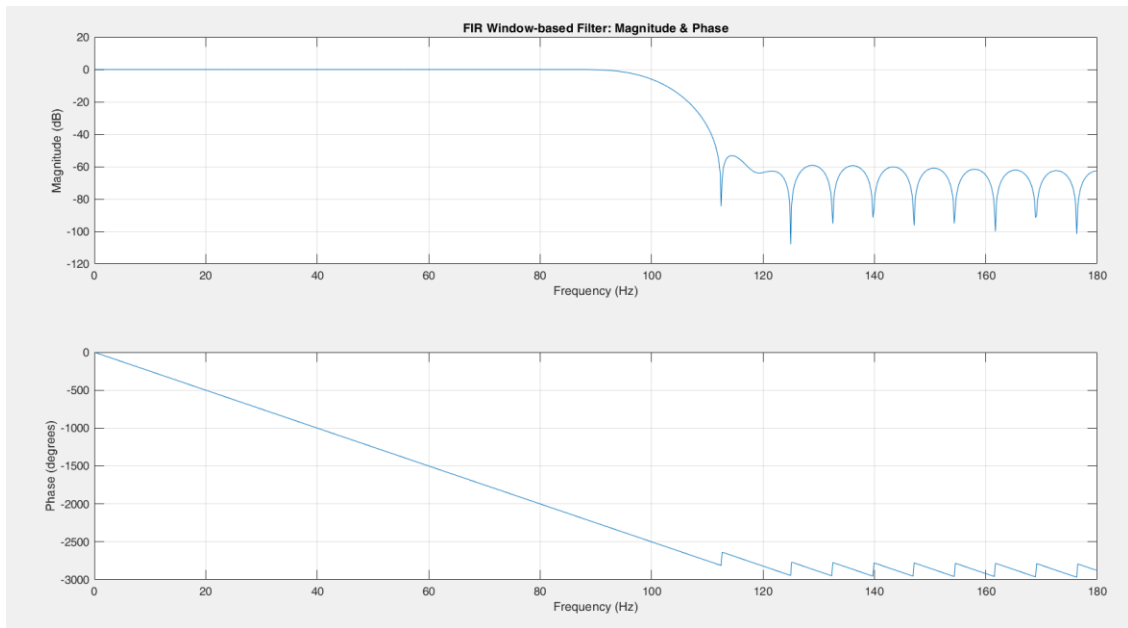


Fig 2.9: Magnitude and Phase Response for FIR Window-based

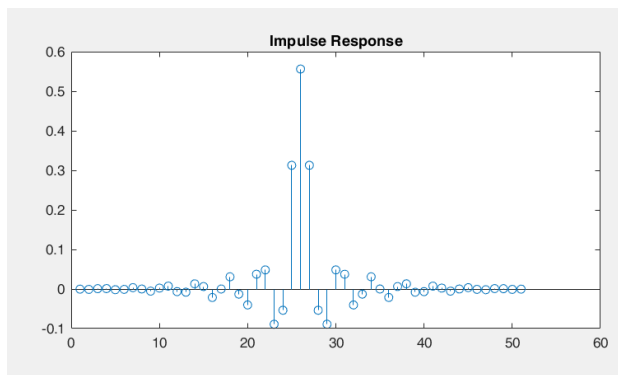


Fig 2.10: Impulse Response for FIR Window-based

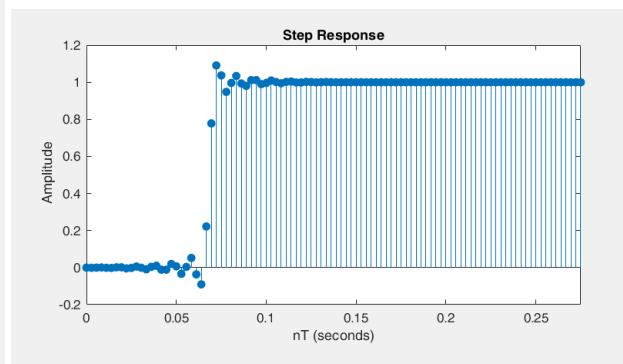


Fig 2.11: Step Response for FIR Window-based

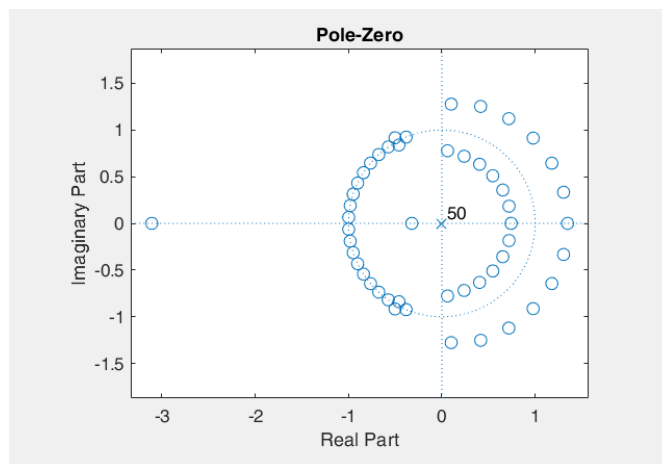


Fig 2.12: Pole-zero diagram for FIR Window-based

MATLAB Code:

```
fs = 360;           % Sampling frequency
fc = 100;           % Cutoff for Muscle Noise
num_taps = 51;      % 50 + 1
b = fir1(num_taps - 1, fc/(fs/2), 'low', hamming(num_taps));

% Analysis
figure;
freqz(b, 1, 1024, fs);
title('FIR Window-based Filter: Magnitude & Phase');

figure;
subplot(2,2,1);
zplane(b, 1);
title('Pole-Zero');

subplot(2,2,2);
stem(b);
title('Impulse Response');

subplot(2,2,3);
stepz(b, 1, 100, fs);
title('Step Response');
```

[3] Implementation and Validation

Now that all the required filters have been designed and implemented individually, they will be combined to accomplish the stated objectives above.

Here is the MATLAB code used to define all the filters in one program, pass the signals of both *Record 100* and *Record 106* through the cascaded filters, and display a few comparison plots to prove the filters effects and the output signals:

NOTE: Consider **ecg100** and **ecg106** as signal data matrixes for Record 100 and Record 106 respectively, and **t** as follows in the variable assignment.

```

fs = 360;                % Sampling frequency

t = (0:length(ecg100)-1) / fs;

% High Pass Filter
fc1 = 0.5;                % Cutoff frequency
order = 4;                % Filter order
[b_hp, a_hp] = butter(order, fc1/(fs/2), 'high');

% Notch Filter
f_notch = 50;             % Frequency of the notch
bw = 2;                   % Bandwidth of the notch
[b_n, a_n] = iirnotch(f_notch/(fs/2), bw/(fs/2));

% Low Pass Filter
fc2 = 100;                % Cutoff for Muscle Noise
num_taps = 51;            % 50 + 1
b_lp = fir1(num_taps - 1, fc2/(fs/2), 'low', hamming(num_taps));

% Filtering signal
% Record 100
ecg100_1 = filtfilt(b_hp, a_hp, ecg100);
ecg100_2 = filtfilt(b_n, a_n, ecg100_1);
ecg100_final = filtfilt(b_lp, 1, ecg100_2);

% Record 106
ecg106_1 = filtfilt(b_hp, a_hp, ecg106);
ecg106_2 = filtfilt(b_n, a_n, ecg106_1);

ecg106_final = filtfilt(b_lp, 1, ecg106_2);

```

Now **ecg100_final** and **ecg106_final** include the filtered output signals for both Record 100 and Record 106 respectively.

The continuation of the MATLAB code below begins plotting all comparison plots and graphs for analysis and validation.

```

% Comparison
figure;
subplot(2,2,1);
plot(t, ecg100, 'r');
title('Record 100 - Raw');
xlabel('Time (s)');
ylabel('mV');
grid on;

subplot(2,2,2);
plot(t, ecg100_final, 'b');
title('Record 100 - Filtered');
xlabel('Time (s)');
ylabel('mV');
grid on;

subplot(2,2,3);
plot(t, ecg106, 'r');
title('Record 106 - Raw');
xlabel('Time (s)');
ylabel('mV');
grid on;

subplot(2,2,4);
plot(t, ecg106_final, 'b');
title('Record 106 - Filtered');
xlabel('Time (s)');
ylabel('mV');
grid on;

% Power spectral density
figure;
subplot(2,1,1);
pwelch(ecg100, [], [], [], fs);
hold on;
pwelch(ecg100_final, [], [], [], fs);
title('PSD Comparison - Record 100');
legend('Raw', 'Filtered');

subplot(2,1,2);
pwelch(ecg106, [], [], [], fs);
hold on;
pwelch(ecg106_final, [], [], [], fs);
title('PSD Comparison - Record 106');
legend('Raw', 'Filtered');

```

```

figure;
subplot(2,1,1);
spectrogram(ecg100_final, hamming(128), 120, 128, fs, 'yaxis');
title('Spectrogram of Filtered Record 100');

subplot(2,1,2);
spectrogram(ecg106_final, hamming(128), 120, 128, fs, 'yaxis');
title('Spectrogram of Filtered Record 106');

% SNR Improvement
noise100 = ecg100 - ecg100_final;
noise106 = ecg106 - ecg106_final;
snr100 = 10 * log10(var(ecg100_final) / var(noise100));
snr106 = 10 * log10(var(ecg106_final) / var(noise106));
fprintf('SNR Improvement Summary: \n');
fprintf('Record 100 : %.2f dB\n', snr100);
fprintf('Record 106 : %.2f dB\n', snr106);

```

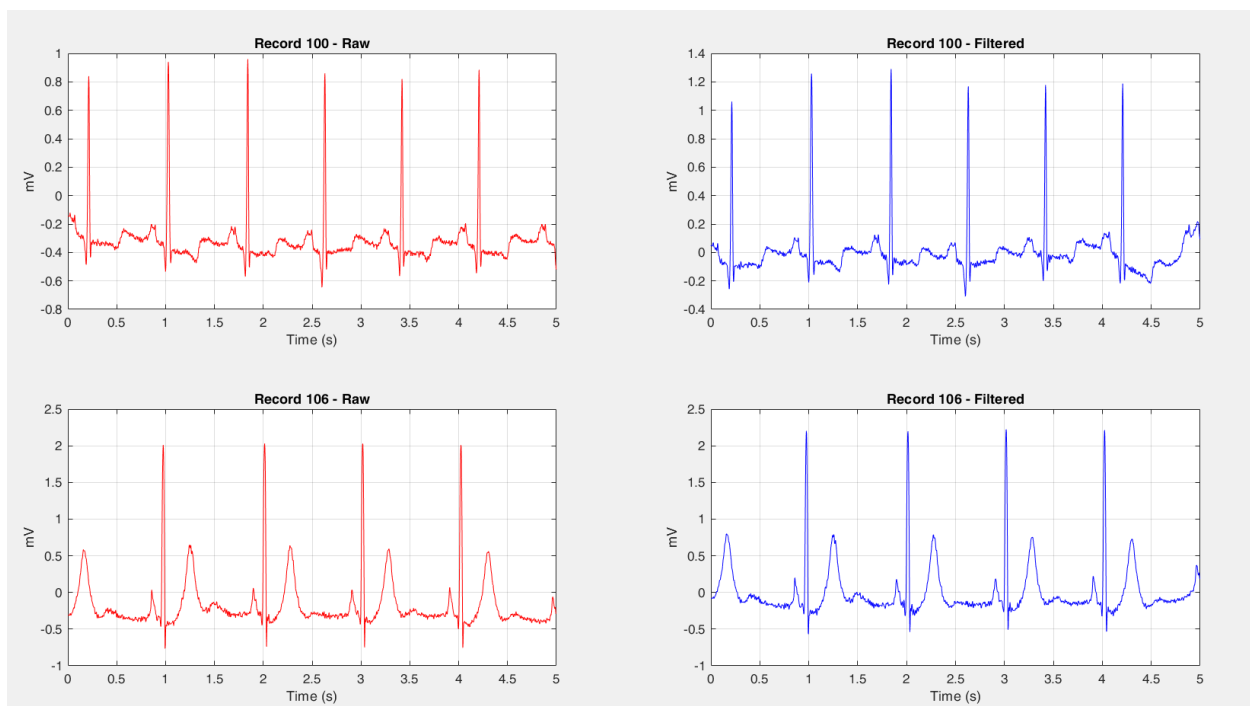


Fig 3.1: Before and after filtering signal plots

```

SNR Improvement Summary:
Record 100 : 9.86 dB
Record 106 : 16.51 dB

```

Fig 3.2: SNR improvement print output

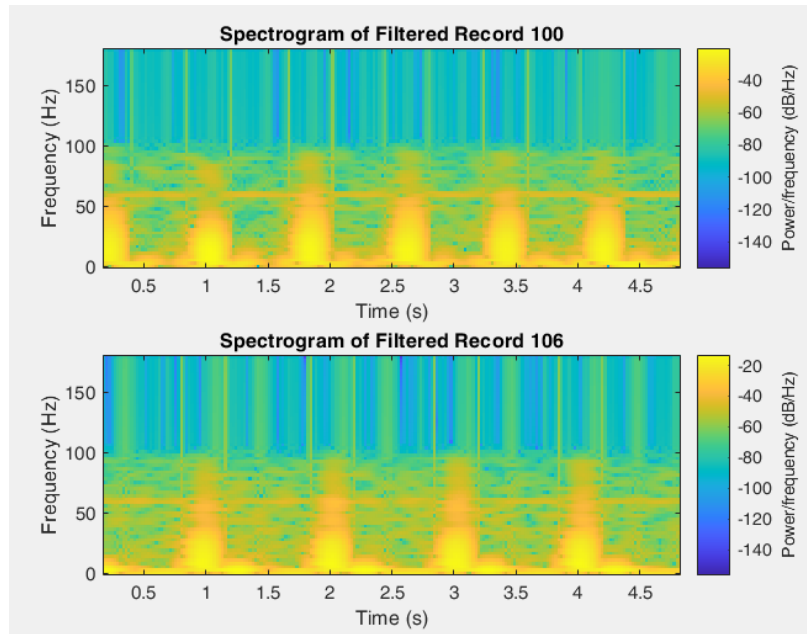


Fig 3.3: Spectrogram of both output signals

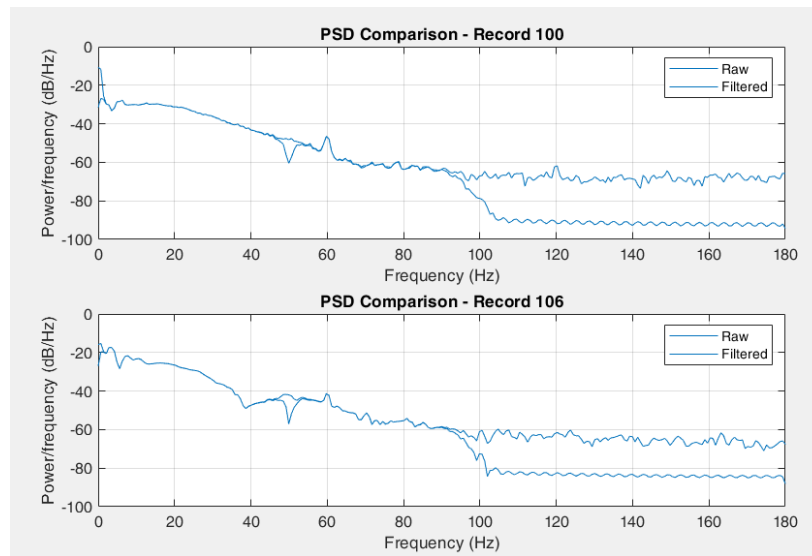


Fig 3.4: Power Spectral Density of both output signals

